

ESP Thread Border Router

The Unit Gateway H2 supports running the ESP Thread Border Router SDK with the ESP32 series Wi-Fi SoC. This SDK is built based on ESP-IDF and OpenThread, running the Thread network on the H2, which communicates with the main processor via a serial port.

1. Preparation

- 1.Environment Configuration: Refer to the [ESP-IDF - ESP32S3 Getting Started Guide](#) to set up the basic compilation environment.

ESP-IDF version

It is recommended to use ESP-IDF version **v5.3.1** for compiling this example.

```
git clone -b v5.3.1 --recursive https://github.com/espressif/esp-idf.git
cd esp-idf
./install.sh
. ./export.sh
```

- 2.Hardware Products Used:

- [Unit Gateway H2](#)
- [CoreS3](#)



- 3.The `idf.py` commands used in the subsequent tutorials depend on ESP-IDF. Before running the commands, you need to execute `./export.sh` in ESP-IDF to activate the relevant environment variables. For detailed instructions, please refer to the [ESP-IDF - ESP32S3 Getting Started Guide](#).

2. Compile RCP Firmware

- 1.Before compiling the Thread Border Router firmware, you need to generate the RCP firmware first. Refer to the following instructions to enter the corresponding RCP firmware directory and set the compilation target to `esp32h2`.

Baud Rate Modification

Due to factors such as the pin drive capability of the chip, the serial port may not work properly when using a long connecting cable with the Unit. If this happens, you need to reduce the speed appropriately. Here, the default baud rate of 460800 is reduced to 230400.

- 2.Modify the baud rate by opening `main/esp_ot_config.h` and changing line 43 `.baud_rate = 460800;` to `.baud_rate = 230400;`.

```
examples > openthread > ot_rcp > main > h esp_ot_config.h > ESP_OPENTHREAD_DEFAULT_HOST_CONFIG
27  #if CONFIG_OPENTHREAD_RCP_UART
37  {
39      .host_uart_config = {
40          .baud_rate = 230400,
41      }
42  }
43  }
44  }
45  }
46  }
```

esp_ot_config.h Git 本地更改(工作树) - 第 1 个更改(共 1 个)

```
40 40      .port = 0,
41 41      .uart_config =
42 42      {
43 43          .baud_rate = 460800,
44 44          .baud_rate = 230400,
45 45          .data_bits = UART_DATA_8_BITS,
46 46          .parity = UART_PARITY_DISABLE,
47 47          .stop_bits = UART_STOP_BITS_1,
```

```
cd examples/openthread/ot_rcp
vim main/esp_ot_config.h
# line 43 .baud_rate = 460800; change to .baud_rate = 230400;
```

- 3. After completing the configuration, execute the following commands to compile the RCP firmware.

```
cd $IDF_PATH/examples/openthread/ot_rcp
idf.py set-target esp32h2
idf.py build
```

- 3. Open the Unit Gateway H2 case, hold down the boot button, and then connect the USB power supply to enter download mode. Execute the following commands to flash the firmware.



```
idf.py flash
```

3. Compile ESP Thread BR Firmware

- 1. Pull the project.

```
git clone https://github.com/Ocean-lhy/esp-thread-br.git
```

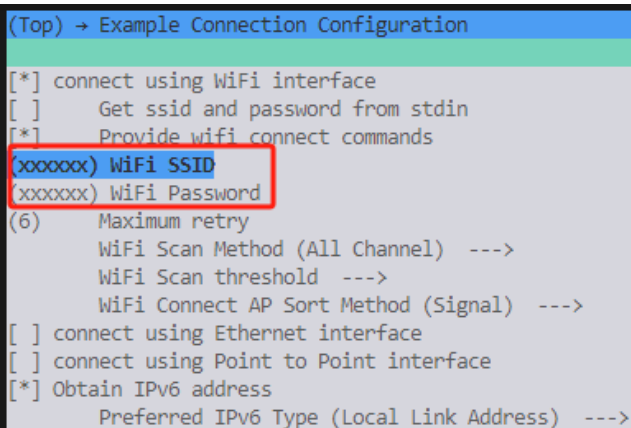
Using Unit Gateway H2 as RCP to implement ESP Thread Border Router requires disabling RCP flashing, modifying the baud rate, and changing the serial port pins. The corresponding code branches have been modified and can be directly switched for use.

- 2. Switch to the corresponding branch based on the main control device used.

```
# coreS3
git checkout demo_for_unit_coreS3
cd examples/thread_border_router_credential_sharing
idf.py set-target esp32s3
# core2 v1.0 and v1.1 have different power management chips, AXP192 and AXP2101 respectively, which need
git checkout demo_for_unit_core2
cd examples/thread_border_router_credential_sharing
idf.py set-target esp32
# core
git checkout demo_for_unit_core
cd examples/thread_border_router_credential_sharing
idf.py set-target esp32
```

- 3. Use `idf.py menuconfig` to enter the configuration page. Configure WiFi information in menuconfig: **Component config** -> **Example Connection Configuration**

```
idf.py menuconfig
```



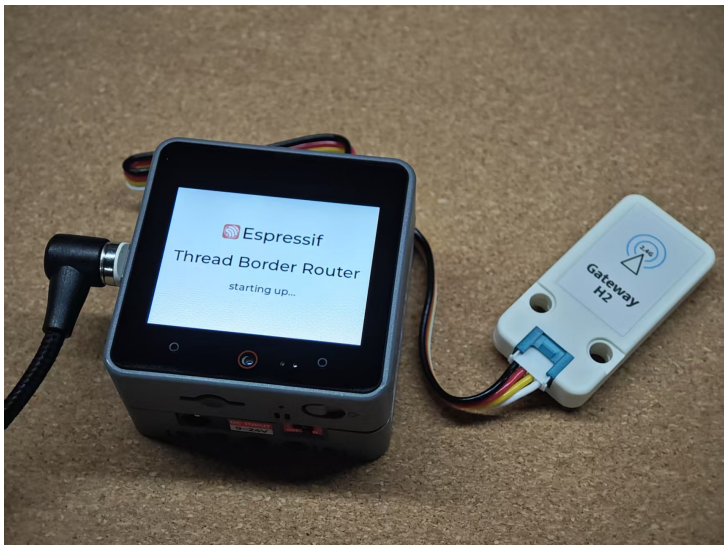
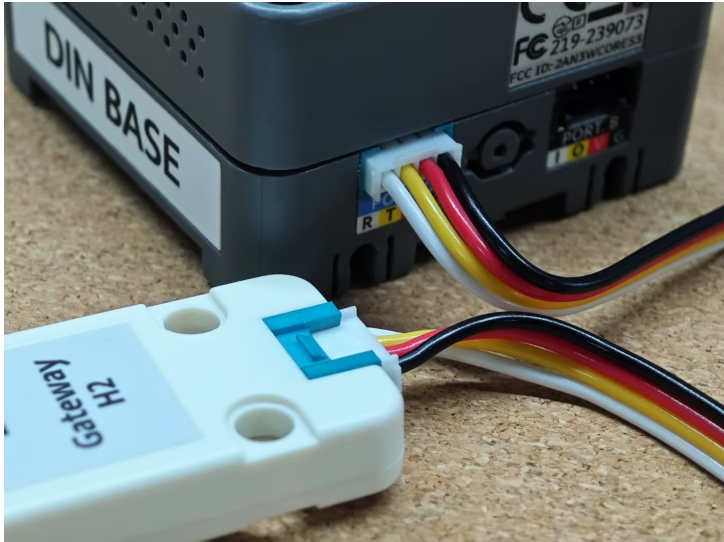
```
(Top) → Example Connection Configuration
[*] connect using WiFi interface
[ ]   Get ssid and password from stdin
[*]   Provide wifi connect commands
xxxxxx WiFi SSID
xxxxxx WiFi Password
(6)   Maximum retry
      WiFi Scan Method (All Channel) --->
      WiFi Scan threshold --->
      WiFi Connect AP Sort Method (Signal) --->
[ ] connect using Ethernet interface
[ ] connect using Point to Point interface
[*] Obtain IPv6 address
      Preferred IPv6 Type (Local Link Address) --->
```

- 4. Compile and flash the ESP Thread BR firmware

```
idf.py build
idf.py erase_flash
idf.py flash
```

4. Start Running

- 1.Connect the Unit Gateway H2 to the main control device PORT.C, and wait for the device to connect to Wi-Fi and the Thread network. After the device initialization is complete, the following information will be displayed:
- **generate epskc** button
- **factoryreset** button
- **Border router web server URL**





- 2.Click the **generate epskc** button, and the device will generate an epskc, which will be displayed on the screen and can be used for quick network access.



- 3.Within the local network, use a browser to access the Border router web server URL to view Thread network information.

ESPRESSIF-OpenThread.

HomeScanFormSettingsStatusTopologyContact

Thread Network Status

The status of the ESP-OpenThread network, including its IPv6, network, OpenThread, RCP, WPAN, and other components, would be displayed here.

OverViewIPv6NetworkOpenThreadRCPWPAN

[IP Address]

Link Local Address
fe80:0:0:0:4419:9064:c6b7:3164

Routing Local Address
fd00:db8:a0:0:0:ff:fe00:e800

Mesh Local Address
fd00:db8:a0:0:ab9a:b249:99c3:b7c5

Mesh Local Prefix
fd00:db8:a0:0:fd00::/68

[Network Information]

Name
OpenThread-ESP

PANID
0x1234

Partition ID
937423390

Extended PANID
dead00beef00cafe

Border Agent ID
b7fd55d84c491f6cd42e75bcd68126

[OpenThread Parameter]

Version
openthread-esp32/9d7f2d69f5-005c5cefc;
esp32s3; 2025-03-13 03:29:22 UTC

Version API
458

Role
leader

PSKc
104810e2315100afd6bc9215a6bfac53

[RCP Information]

Channel
21

IEEE EUI-64
4831b7ffec8d160

TxPower
20 dBm

Version
openthread-esp32/9d7f2d69f5-005c5cefc; esp32h2; 2025-03-13 02:45:07 UTC

[WPAN Status]

Service
associated

- ## 5. Testing

1. In the ThreadBorderRouter background, input `networkkey`, `panid`, `channel` to obtain the Thread network's network key, panid, and channel.
2. In the SimpleCLI example, input configuration commands and start the Thread network

3. In the SimpleCLI serial interaction, input **state** to view the Thread network status. If connected as a child/router, the Thread network connection is successful. If established as a leader, the configuration may be incorrect.
4. In the SimpleCLI serial interaction, input **parent** to view the Thread network's parent node; input **extaddr** to view the extended address of this node.
5. In the ThreadBorderRouter serial interaction, input **extaddr** to view the extended address of this node, which should match the **parent extaddr** in the SimpleCLI example.
6. In the ThreadBorderRouter serial interaction, input **neighbor table** to view the Thread network's neighbor nodes, which should include the node from the SimpleCLI example.

6/6 | Update Time: 2026-04-28